

Xolaris LOE — Remaining Work after 3 Months

Autonomous Security (Xolaris) · 9 September 2026 · 1 engineer · 6 productive hours/day
Source: docs/architecture modules 0–22, adapter guide, Feature 4.5 Reporting, Feature 4.6 Findings/Assets APIs.
Hours are engineering time only. GPU rental and Hugging Face download wait are extra.

3 mo Code already built	15 Modules done	10 Partial / not hosted	83d Remaining person-days
-----------------------------------	---------------------------	-----------------------------------	-------------------------------------

What is left is mostly hosting, not new engines. The control plane is written: Scan → Evidence → Trust → Risk → Decision → Plan → Simulation → Approval → Execution → Verification → Reporting. What is not done is putting RavenX + llama.cpp + scanner binaries + Postgres on a Linux cloud box, then proving the full API path and locking down tenants. Without that, the product only runs in simulate mode on a laptop with SQLite and in-memory chat.

1. Delivery mix — code vs production

Share of platform readiness (not lines of code): ~40% production-ready.

- 40% engines/APIs coded
- 25% partial (OPA, threat-intel feeds, live scanners, UI)
- 20% cloud / model / database / memory
- 15% auth, E2E tests, pipeline glue, RAG

Remaining effort: **83 person-days / 490 hours** (~17 calendar weeks at 5 days/week, one person).

Wave	Focus	Days	Hours
A Host	Cloud + RavenX + llama.cpp + scanners	10	62
B Data	Postgres + memory + E2E	17	96
C Product	Pipeline + auth + UI	18	106
D Later	Intel, RAG, real execute	38	226
Go-live minimum (A + B)		27	158
Total remaining		83	490

2. Already built (past 3 months) — do not rebuild

#	Feature / module	Status	What exists
0	Platform / FastAPI app	Done	Chat, health, CORS, SQLite default
1	Decision Engine	Partial	Wired to chat; ScanAware router for “scan ...”; default still CHAT
2	Context Manager	Partial	Wired to LLMService V2; RAG builder still empty
3	Prompt Builder	Partial	Wired; RAG/Agent templates are placeholders
4	Provider Factory + LlamaProvider	Partial	Llama generate works; OpenAI/Anthropic not implemented

5	Canonical Security Objects	Done	Findings, evidence, decisions, remediations
6	Normalization Service	Done	Nuclei, Trivy, Prowler, Checkov, Gripe
7	Policy Engine	Partial	Local rules fail-closed; OPA transport stub
8	Tool adapters	Partial	Nuclei, Trivy, Prowler, Checkov, Gripe code; binaries not hosted
9	Evidence Repository	Done	Persist, dedupe, search + GET /findings (Feature 4.6)
10	Asset Inventory	Done	Catalog + GET /assets (Feature 4.6)
11	Threat Intelligence	Partial	Enrichment services exist; NVD/KEV/MITRE/MISP feeds skipped
12	Trust Scoring	Done	Deterministic + REST
13	Risk Engine	Done	Deterministic + REST
14	Decision Service	Done	Deterministic advisor; AI optional flag off
15	Remediation Planner	Done	Plans + REST
16	AI Harness	Done	Governance wrapper + REST
17	Simulation Engine	Done	Dry-run + REST; not auto-chained after plan
18	Approval Engine	Done	REST; not auto-submitted after simulation
19	Execution Engine	Partial	Orchestration done; adapter records success, no real infra
20	Verification Engine	Done	Post-remediation checks + REST
21	Reporting & Analytics	Done	Feature 4.5.4 validation/stabilization
22	Scan / Ingest	Partial	API + simulate path; live needs scanner binaries on Linux
4.6	Findings & Assets HTTP	Done	Search/get/history; shared store with scan
UI	React prototype	Partial	Screens exist; still mock data, not calling FastAPI

3. HTTP already mounted

POST /api/v1/scans · GET /api/v1/findings · GET /api/v1/assets · trust · risk · decisions · remediation-plans · simulations · approvals · executions · verifications · reporting · ai-harness · /chat

Missing HTTP: threat-intel list/get, authenticated user, scan job status, persisted chat sessions.

4. Remaining LOE by work item

ID	Work item	Days	Hours	Why it is still open
A1	Cheap GPU Linux cloud (Vast.ai / RunPod / Lambda)	2	12	Need a Linux box with GPU/RAM to host RavenX. Vast.ai is usually cheapest for hourly GPU.
A2	Download RavenX from Hugging Face	1	6	Get the GGUF/weights onto the VM disk. Time is mostly download + checksum.
A3	Build and configure llama.cpp server	2	12	Compile llama.cpp, serve OpenAI-compatible /v1/chat/completions on port 8080. Backend already calls LLAMA_BASE_URL.
A4	Deploy backend + pip install requirements	1	8	Clone repo, Python venv, requirements.txt, .env (LLAMA_BASE_URL, DATABASE_URL), uvicorn.

A5	Install live scanners on Linux	3	18	Nuclei, Trivy, Prowler, Checkov, Gripe binaries + PATH. Today scans only work in simulate mode without these.
A6	Smoke: live scan + chat to RavenX	1	6	POST /scans mode=live, /chat against llama.cpp, confirm findings appear on GET /findings.
B1	PostgreSQL instead of SQLite	3	18	docker-compose.yml already has Postgres 16. Point DATABASE_URL, create DB, CREATE_TABLES or migrations.
B2	Alembic migrations (production schema)	3	16	CREATE_TABLES is fine for demo; production needs versioned migrations across all engine tables.
B3	Multi-tenant persistent storage hardening	3	16	Engines already require tenant_id. Left: backups, indexes, connection pool, no SQLite file, isolate tenants in ops.
B4	Persistent chat memory	3	16	MemoryService is in-RAM (max 20 messages). Restart wipes chat. Store sessions in Postgres per tenant.
B5	End-to-end API testing	5	30	Full path: scan → findings → trust → risk → decision → plan → simulate → approve → execute → verify → report. Unit tests exist per engine; live E2E does not.
C1	Auto-chain pipeline after Plan	4	24	FindingPipeline stops at Planner. Simulation, Approval, Execution, Verification exist but must be called separately.
C2	API authentication (JWT or API keys)	5	30	Every route is open. Needed before any cloud demo on a public IP.
C3	Wire React UI to real APIs	6	36	Frontend is mock. Replace with GET /findings, /assets, trust, risk, plans, approvals, /chat.
C4	Async scan jobs + GET /scans/{id}	3	16	Live Nuclei/Trivy can run minutes. Today the HTTP call waits until the scan finishes.
D1	Threat intel REST + one live feed (CISA KEV)	5	30	Module is placeholder feeds only. No GET /threat-intel. KEV is the smallest real connector.
D2	OPA policy transport	3	16	Local policy works. OPA backend is fail-closed stub.
D3	OpenAI / Anthropic providers	2	12	Placeholders. Only Llama/RavenX path works.
D4	RAG / document grounding	8	48	RAGBuilder returns no documents. Needed for “ask about our findings” with real evidence.
D5	Real execution adapters (cloud/SSH)	8	48	Execution engine records fake success. Production needs provider adapters without changing orchestration.
D6	Extra scanners (Gitleaks, ZAP, OSV, Semgrep)	6	36	Adapter guide lists these as next tools. Pattern already exists (~1 day each).
D7	MCP agent tools + logging/HTTPS	6	36	Docs list get_finding, decide, approve as next MCP tools. Also nginx TLS and structured logs.

5. Wave summaries

Wave A — go-live host (must) · 10 days · 62 hours

Buy a cheap Linux GPU VM (Vast.ai first, then RunPod if GPUs are scarce). Download RavenX from Hugging Face. Build llama.cpp and serve it on 127.0.0.1:8080. Clone backend, pip install, set LLAMA_BASE_URL and DATABASE_URL. Install Nuclei, Trivy, Prowler, Checkov, Gripe. Run one live scan and one chat. Plus GPU rent (~USD 200–500/month if left on).

Wave B — data that survives restart (must) · 17 days · 96 hours

Switch SQLite to PostgreSQL (compose file already in backend/docker-compose.yml). Add migrations. Confirm tenant_id isolation on findings/assets/trust/risk. Replace in-memory MemoryService with Postgres chat history. Run end-to-end API tests through the whole pipeline.

Wave C — product (should, after host is up) · 18 days · 106 hours

Auto-run Simulation and submit Approval after a plan. Add login or API keys before the VM is public. Point the React screens at real findings/assets/chat. Make live scans async so Nuclei does not block HTTP.

Wave D — later (do not start before A+B) · 38 days · 226 hours

Live KEV/NVD threat feeds, OPA, OpenAI/Anthropic fallback, RAG over findings, real patch/SSH execution (today it only records success), extra scanners (Gitleaks, ZAP, OSV), MCP tools, HTTPS and logging.

6. Suggested calendar (one engineer)

Block	When	Days	Outcome
A Host	Weeks 1–3	10	RavenX answering chat; live Nuclei/Trivy scan writing findings
B Data	Weeks 3–6	17	Postgres + tenant storage + chat memory + E2E test pack
C Product	Weeks 6–10	18	Pipeline past Plan; auth; UI on real APIs
D Later	Weeks 10–16	38	Live intel, RAG, real execute, extra tools

Planner mapping. Keep Feature 4.5 Reporting and 4.6 Findings/Assets as complete. Next feature bucket should be 4.7 Cloud Hosting (Wave A) then 4.8 Data Plane (Wave B). Do not open Wave D stories until A and B are done — the engines are already written.

Assumptions: 1 engineer; 6h/day; Vast.ai/RunPod GPU Linux; RavenX GGUF served by llama.cpp; scanners installed as OS packages; Postgres 16; no parallel team. Cloud invoice is extra (GPU hourly). Dates are relative to the start of Wave A, not to the 3 months already spent.